

PNP USER GUIDE

Document Status	Approved
-----------------	----------

ANALOG DEVICES, INC.

www.analog.com

Table 1: Revision History

[illegible]

Copyright, Disclaimer Statements

Copyright Information

Copyright (c) 2023 Analog Devices, Inc. All Rights Reserved. This software is proprietary and confidential to Analog Devices, Inc., and its licensors. This document may not be reproduced in any form without prior, express written consent from Analog Devices, Inc.

Disclaimer

Analog Devices, Inc. reserves the right to change this product without prior notice. Information furnished by Analog Devices is believed to be accurate and reliable. However, no responsibility is assumed by Analog Devices for its use; nor for any infringement of patents or other rights of third parties which may result from its use. No license is granted by implication or otherwise under the patent rights of Analog Devices, Inc.

Table of Contents

Revision List.....	2
Copyright, Disclaimer Statements	3
Table of Contents.....	4
List of Figures	6
List of Tables.....	7
1 Introduction.....	9
1.1 Purpose	9
1.2 Assumption.....	9
1.3 Organization of the document.....	9
1.4 Terminology.....	9
2 Plug and Play Overview.....	10
2.1 Brief Introduction.....	10
2.2 PnP System and Requirements	10
2.2.1 Requirements	10
2.3 High Level Architecture.....	11
3 Example Platforms and Application	12
3.1 Package Details.....	12
3.2 Platforms	13
3.2.1 Windows/PC Application	14
3.3 SC594	15
3.3.1 RJ45	15
3.3.2 XLR.....	16
3.4 Using Windows Terminal Application.....	16
3.4.1 LibUSB driver for USBi.....	17
3.4.2 Using Commands	17
3.4.3 Batch Execution of Commands	18
3.5 Using ADSP-SC59x UART Terminal Application	20
4 Development Workflow.....	21
4.1 SigmaStudio+ Usage for A2B and PnP.....	22

5 Building PnP for Custom Platform.....	25
5.1 I2C.....	25
5.2 SPI	25
5.3 Timer	25
5.4 Audio Host.....	25
5.5 Plugins and Memory Management	26
5.6 Operating System Consideration	26
5.6.1 Linux and ALSA	26
5.7 Compile Time Parameters	26
5.7.1 Header files.....	26
5.7.1.1 feature.h.....	26
5.7.1.2 conf.h	26
5.7.2 Bus Configuration File.....	27
5.7.3 Memory.....	27
6 Application Integration	28
6.1 Overview of Application and PnP Interaction	28
6.2 PnP Module Initialization.....	31
6.3 Periodic Tick.....	31
6.4 Network Control and Status	32
6.5 Audio Routing	32
6.5.1 Audio Groups.....	32
6.5.2 Audio Streams	33
6.5.3 Apply Routing	34
6.5.4 Impact of Network Events on Audio Routing	35
6.5.5 Application Call-backs.....	36
6.6 Data tunnel	36
6.7 GPIO over Distance.....	37
6.8 Callbacks.....	38
6.8.1 PNP CALLBACK.....	38
6.8.2 ERROR CALLBACK	38
6.8.3 INTERRUPT CALLBACK.....	39
6.9 Multi-Main Considerations	39
7 System Details for ADSP-SC59x Platform.....	42
7.1 System Requirements	42
7.1.1 Software Requirements.....	42
7.1.2 Hardware Requirements	42
7.2 Hardware Setup - XLR.....	42

7.2.1 Evaluation boards	42
7.2.2 EVAL-AD2437A1NZ	42
7.2.3 EVAL-AD2437B1NZ	44
7.2.4 EV-SC594-SOM Rev B	44
7.3 Connections.....	45
7.3.1 Evaluation boards	45
7.3.2 USBi	46
7.3.3 Audio In/out.....	46
7.4 Running the Plug and Play Demo - XLR	47
7.4.1 Write PnP Content on the sub nodes using SigmaStudio+	47
7.4.2 Running the demo.....	48
7.4.2.1 HW Setup.....	48
7.4.2.2 Running the Target Demo application	48
7.5 Considerations for RJ45 Evaluation board Set up.....	50
7.5.1 Hardware Setup – RJ45.....	51
7.5.1.1 Evaluation boards.....	51
7.5.1.1.1 EVAL-AD2437A1MZ.....	51
7.5.1.1.2 EVAL-AD2437B1MZ.....	52
7.5.1.1.3 EV-SC594-SOM Rev B + EV-SOMCRR-EZKIT	52
7.5.1.1.4 EV-SOMCRR-EZKIT Rev A.....	53
7.5.2 Connections.....	54
7.5.2.1 Evaluation boards.....	54
7.5.2.2 USBi.....	55
7.5.2.3 Audio In/out	55
8 Terminology	56
9 References.....	57

List of Figures

<i>Figure 1: Comparison between Predefined and Dynamic A2B Network.....</i>	<i>10</i>
<i>Figure 2: PnP Architecture.....</i>	<i>11</i>
<i>Figure 3: PnP Supported Platforms</i>	<i>14</i>
<i>Figure 4: Windows PnP Setup</i>	<i>15</i>
<i>Figure 5: SC594 RJ45 PnP Setup</i>	<i>15</i>
<i>Figure 6: SC594 XLR PnP Setup.....</i>	<i>16</i>
<i>Figure 7: help Command.</i>	<i>17</i>
<i>Figure 8:Initial commands</i>	<i>18</i>

Figure 9: EEPROM Update Commands.....	19
Figure 10: Evaluation Commands.....	19
Figure 11: Development Workflow for PnP.....	21
Figure 12: SigmaStudio+ schematics	22
Figure 13: SigmaStudio+ Windows.....	22
Figure 14: Write PnP Content using Sigma Studio+	23
Figure 15: App-PnP State Flow	29
Figure 16: High Level PnP Flow	30
Figure 17: Periodic Tick.....	32
Figure 18: Dynamic Audio Routing	33
Figure 19: addCon Command	34
Figure 20: Audio Routing Limitation.....	35
Figure 21: Data Tunnel Topology	36
Figure 22: Data Tunnel Configuration.....	37
Figure 23: addTunnel Command.....	37
Figure 24: GPIO over Distance.....	37
Figure 25: Multi-Main Design.....	39
Figure 26: Multi-Main Process.....	40
Figure 27: start Command.....	40
Figure 28: setNetwork Command	41
Figure 29: Multi-main Config File.....	41
Figure 30: A2B EVAL-AD2437A1NZ board	43
Figure 31: A2B EVAL-AD2437B1NZ board	44
Figure 32: EV-SC594-SOM Rev B (Top view)	45
Figure 33: Eval Board Connections	46
Figure 34: Write PnP Content using Sigma Studio+	47
Figure 35: UART Output - Connection.....	49
Figure 36: UART Output - Disconnection	50
Figure 37: A2B EVAL-AD2437A1MZ board	51
Figure 38: A2B EVAL-AD2437B1MZ board.....	52
Figure 39: EV-SOMCRR-EZKIT + EV-SC594-SOM	54
Figure 40: Eval Board Connections	55

List of Tables

Table 1: Revision History.....	2
Table 2: Jumper Settings.....	43
Table 3: Terminology.....	56

Table 4: References.....57

1 Introduction

This document guides the user to integrate A2B Plug and Play software module to a custom application and Platform. Initially, document describes the example application and platform. Later sections cover porting and integration aspects.

1.1 Purpose

This document enables the user to evaluate the Plug and Play feature of A²B and integrate into the end products.

1.2 Assumption

Document assumes prior knowledge of using SigmaStudio+ tool, embedded software development and overview of A2B technology.

1.3 Organization of the document

Sections 2 provides a brief overview of Plug and Play feature.

Section 3 describes the example application and features to evaluate the module in the example Platform.

Section 4 provides high level workflow for customization. Section 5 and 6 covers porting and integration respectively.

Section 7 provides detailed set up instruction for ADSP-SC59x platform with A2B.

1.4 Terminology

Throughout the document, “Plug and Play software module” is simply referred as PnP. Also, this document uses standard A2B terminologies.

2 Plug and Play Overview

2.1 Brief Introduction

Plug and Play feature is specifically designed for A2B devices for dynamic networking use-cases. Using this feature, A2B devices can be connected dynamically in the existing network and can work seamlessly without re-discovery of the complete network. Importantly, the Main node MCU need not have any prior knowledge about the network and devices.

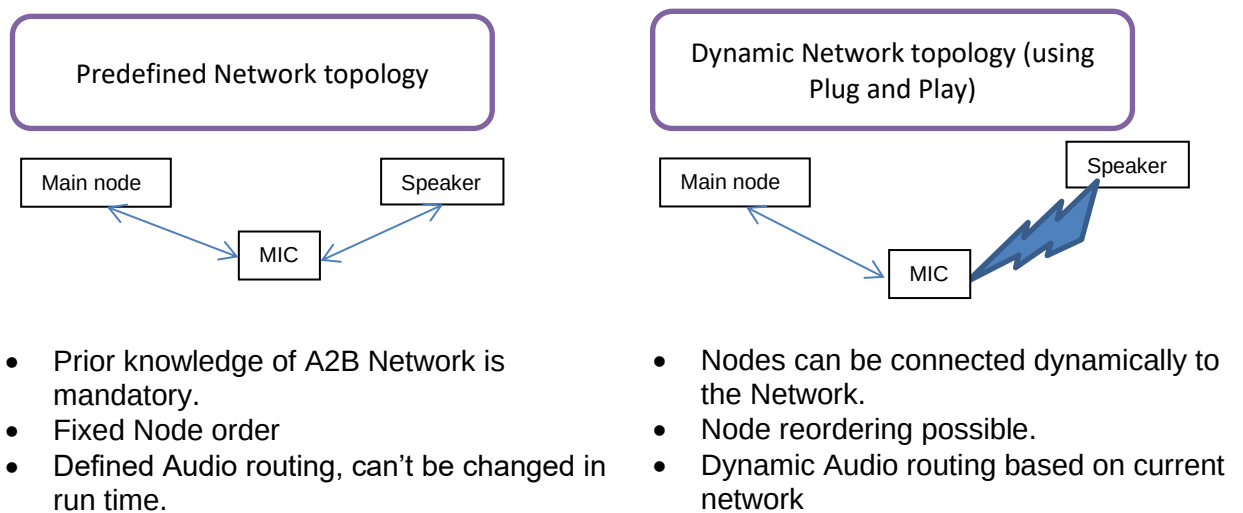


Figure 1: Comparison between Predefined and Dynamic A2B Network

2.2 PnP System and Requirements

In A2B-PnP, every sub node that becomes a part of an existing network carries self-configuration information using an external non-volatile memory (EEPROM). The host MCU accesses the remote EEPROM and configures the node into the network. The Main node configuration is coupled to the host MCU having its settings available in a Bus Configuration File (BCF).

2.2.1 Requirements

- Sub Node: An **EEPROM** on Sub Node is a must have for PnP.
 - I2C programmable
 - 2 Byte Addressing

- 7-bit Slave Address range- "0x50-0x53"
- Size: Minimum 4kB (actual size varies with number of peripherals and the peripheral information)
- Main node: **Bus Config File** holding Main node configuration details.
 - This file becomes part of Host firmware and configures Main node on startup.
- Sub node can be either **local or bus powered**.
 - **Two step discovery** is preferred but not mandatory.

2.3 High Level Architecture

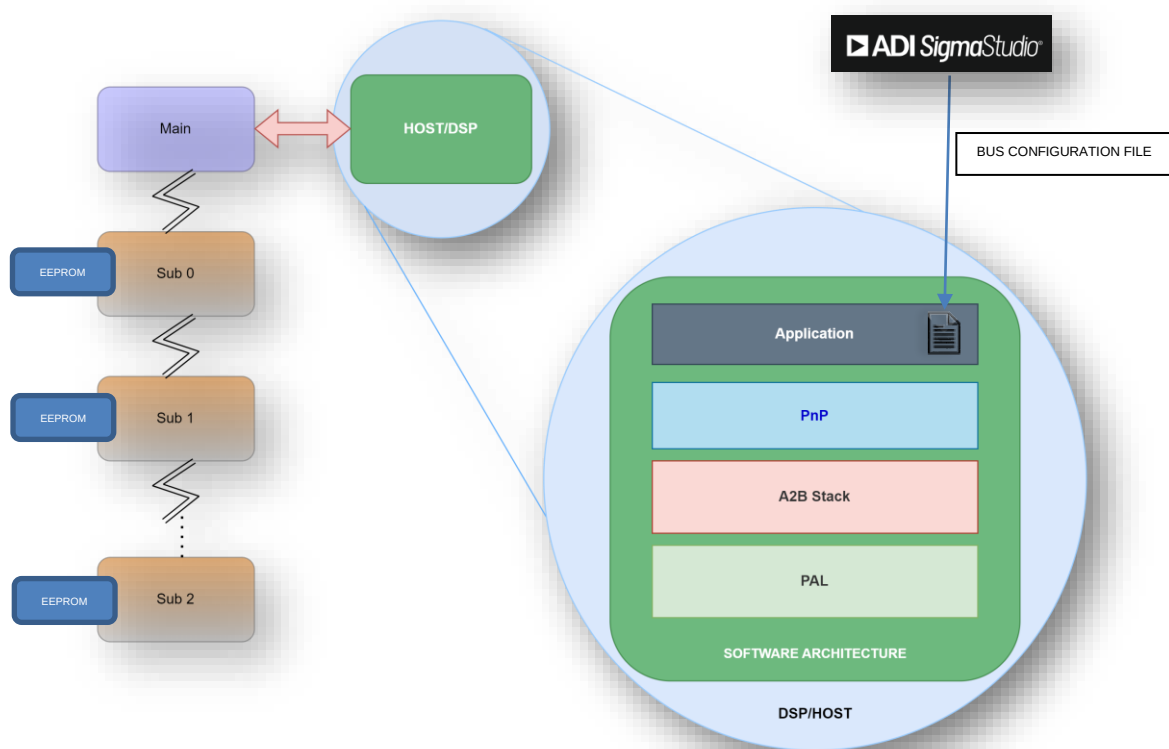


Figure 2: PnP Architecture

This picture shows how the Host MCU handles various system tasks, like setting up the main node, using PnP and A2B software, and reading data from sub-nodes' EEPROMs.

3 Example Platforms and Application

3.1 Package Details

This section talks about the package components in a tree view:

```

| 2023-08-17-PNPSW4A2B Click Thru ELA.pdf - License Agreement
| GettingStarted.rtf - Getting Started Note
|
|— Docs - Documentations
|   A2B_PnP_ContentSpec.pdf
|   A2B_PnP_UserGuide.pdf
|   A2B_PnP_QuickStartGuide.pdf
|   A2B_PnP_API_Reference.chm
|
|— Schematics - SigmaStudio+ Example Schematics for PnP
|   └─ PC
|       | adi_a2b_3node_AD2437A1MZ_PnP.ssprj
|       | adi_a2b_3node_AD2437A1NZ_PnP.ssprj
|       | adi_a2b_multi-main.ssprj
|       └─ xml
|— Target - Software for Different Evaluation Platforms
|   └─ lib
|       └─ software
|           └─ a2bstack - A2B Stack Library
|               └─ app - Application Software
|                   └─ common - Common Software
|                       └─ target-demo - SC594 Demo Projects
|                           └─ a2b-adsp-sc59x_AD2437-RJ45_PnP - RJ45 Project
|                               └─ a2b-adsp-sc59x_AD2437-XLR_PnP - XLR Project

```

```
| | | └─pal-app-pnp - PAL and PnP Libraries
| | |   └─sc59x
| | └─win32 - Windows Demo Project
└─bin - Windows Evaluation Binaries
    └─app
        └─win32
            A2BPnPConsoleApp.exe
            libA2BPnPConsoleApp.dll.a
            libusb-1.0.dll
            run_EEPROM_RJ45.bat
            run_EEPROM_XLR.bat
            run_EVAL_RJ45.bat
            run_EVAL_XLR.bat
        |
    └─cfg - Configuration Files
└─lib - Windows Dependent Libraries
```

3.2 Platforms

Figure 3 displays the various platforms supported in the current release. The PnP and A2B stack work independently of the platforms in use, while platform-specific drivers are tailored to platforms. At the application layer, Windows/PC application utilizes a terminal for user interaction, whereas ADSP-SC59x platforms uses UART terminal.

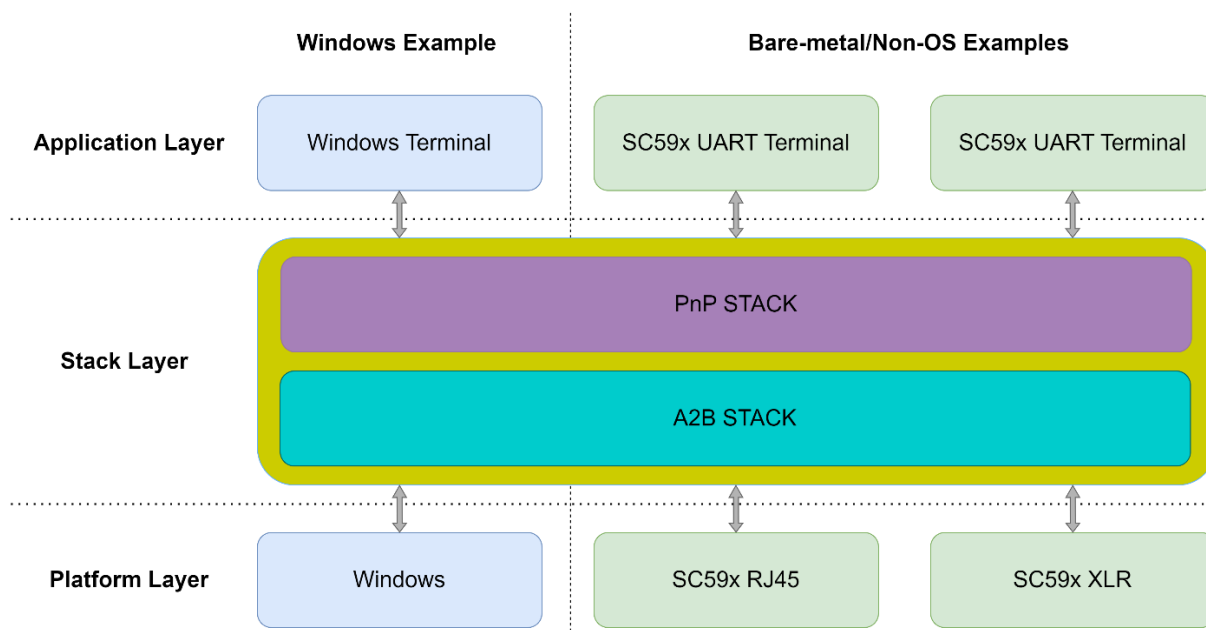


Figure 3: PnP Supported Platforms

3.2.1 Windows/PC Application

Windows evaluation of the release package can be carried out easily by having the below hardware:

- Windows PC (Win10 and above).
- AD2437A1MZ
- AD2437B1MZ
- USBi adapter
- USB to Micro USB B cable
- Audio source and sinks

Figure below shows the hardware setup for Windows evaluation:

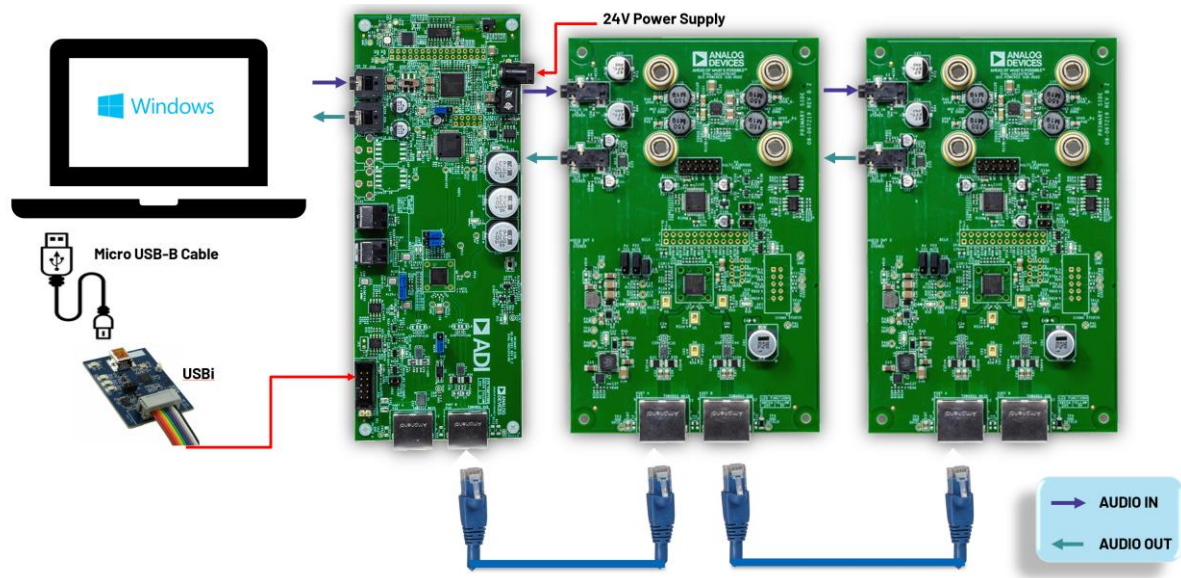


Figure 4: Windows PnP Setup

3.3 SC594

The following figures captures the SC594 platform PnP evaluation. Refer to Section 7 for details

3.3.1 RJ45



Figure 5: SC594 RJ45 PnP Setup

Line OUT

Line IN

XLR Cables

B-Side

A-Side

AD2437 A²B Main board

EV-SOM-SC594

AD2437 A²B Sub board

AD2437 A²B Sub board

AD2437 A²B Sub board

AD2437 A²B Sub board

3.4 Using Windows Terminal Application

This Application talks to the A2B hardware and provides an interface between user and the hardware. It uses USBi device to transmit and receive data between Windows PC and A2B hardware. USBi directly connects to the A2B Main node making the PC as a Host MCU.

This application *A2BPnPConsoleApp.exe*, has all the software stacks (Application, PnP, A2B and PAL as shown in Figure here) ported and compiled for running on Windows environment. On the Application layer three separate threads are spawned for purposes of reading from terminal, writing to terminal and handling PnP interfacing. Threads collaborates with each other to complete the system level jobs.

3.4.1 LibUSB driver for USBi

Terminal application uses LibUSB driver for USBi. Note that this driver is different from the driver used within SigmaStudio+. Please refer to “A2B_PnP_QuickStartGuide.pdf” for quick understanding.

3.4.2 Using Commands

There is a comprehensive set of commands for the user to interact with the A2B network. On start-up of the Windows terminal application, an interactive console is presented to the user. The user communicates with the A2B-PnP network by issuing a specific set of commands in the correct sequence.

To see the list of all the supported commands, ‘help’ command should be entered. Here is an example of ‘help’ command given from application console.

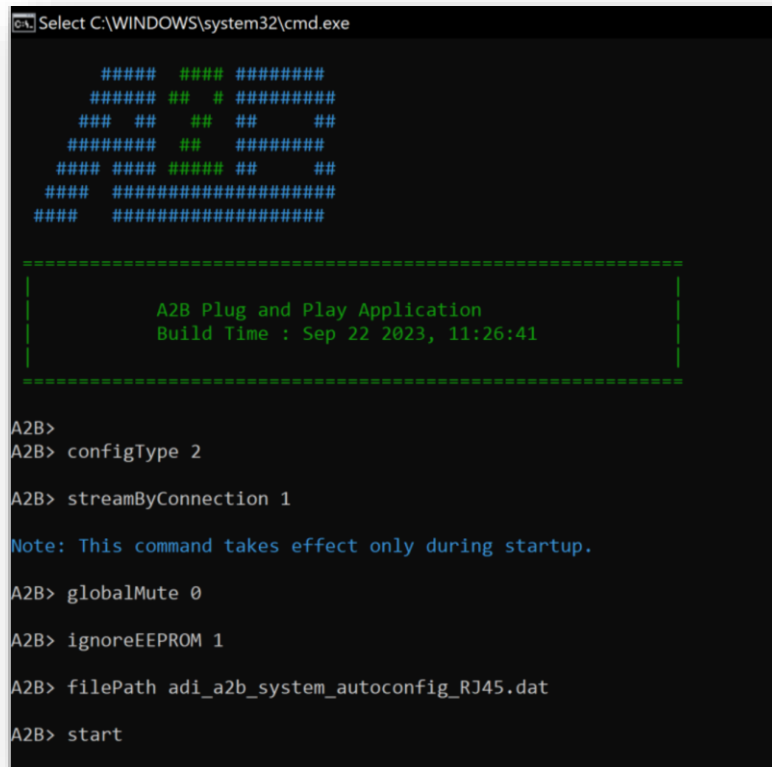
```
A2B> help

help / ?                               Prints this list
addCon <source node: group> <dest node: group>    Adds new connection by position
addStream <source node: group> <dest node: group> <Name>    Add Audio stream(Destinations can be
addTunnel                                         Add new tunnel
applyRouting                                     Apply new audio and tunnel Routing
clear                                             Clears the display
clearStream                                       Clears the streamlist
clearTunnel                                       Remove tunnel
configType <type>                                Main node Configuration source. 0 - BCF, 1 - EEPROM, 2 - FILE
discInterval <value>                             Update discovery interval value
disDbg                                           Disable debug logs
enDbg                                           Enable debug logs
filePath <filename>                             Name of .dat file for configuration (optional)
getBw                                           Get bandwidth of network
getDiscTime <nodeNum>                           Discovery time of a node
getNwInfo                                       Gets Network Info
getNodeInfo <nodeNum>                           Gets node level Info
getVendorID <nodeNum>                           Get Vendor ID
globalMute <0/1>                                Mute globally in case of stream reconfiguration
ignoreEEPROM <0/1>                              Ignore reading of EEPROM
intrptPollInterval <value>                     Update Interrupt Polling interval
maxNumNodes <value>                             Update maximum number of sub nodes
pause                                           Pauses PnP discovery
quit                                            Quit this application
resume                                          Resumes the PnP discovery
rdm <regAddr>                                    Reads the value from the specified A2B register (Main)
rds <nodeNum> <regAddr>                         Reads the value from the specified A2B register (Sub)
setNetwork <nw ID>                              Set the default network to execute further commands
start                                           Starts the PnP application and network discovery
streamByConnection <0/1>                       Enable creating streams by connections
updateEEPROM <filename> <nodeNum> <EEPROM Addr>    Update EEPROM of a node
updateEEPROMall <filename> <EEPROM Addr>          Update EEPROM of all the nodes
ver                                             PnP Software VersionA2B>
wrm <regAddr> <Val>                             Writes the specified value into the given A2B register (Main)
wrs <nodeNum> <regAddr> <Val>                   Writes the specified value into the given A2B register (Sub)
```

Figure 7: help Command.

Few of the commands are intended for initial configuration and should be provided before triggering the PnP operation.

Shown below are few of those commands and their usage.



```

Select C:\WINDOWS\system32\cmd.exe

#####  #####
#####  ##  #####
###  ##  ##  ##
#####  ##  #####
####  #####  ##
####  #####
####  #####

=====
|
|          A2B Plug and Play Application
|          Build Time : Sep 22 2023, 11:26:41
|
|=====

A2B>
A2B> configType 2

A2B> streamByConnection 1

Note: This command takes effect only during startup.

A2B> globalMute 0

A2B> ignoreEEPROM 1

A2B> filePath adi_a2b_system_autoconfig_RJ45.dat

A2B> start

```

Figure 8:Initial commands

3.4.3 Batch Execution of Commands

To ease out the evaluation process, few batch files are provided at *Targets/software/bin/app/win32/*. Someone using a brand-new sub node evaluation hardware or a hardware with no EEPROM data should double click on `run_EEPROM_RJ45.bat`. This file updates the EEPROM of each sub node connected in the network and makes it ready for further execution. A default module configuration file (EEPROM data) has been provided in the package (*Targets/software/cfg*) to start with.

This batch file executes the application along with a series of preset commands provided in a text file at *Targets/software/cfg/init_EEPROM_RJ45.txt*. Here is a snip of the file contents.

```
1 %enDbg
2 configType 2
3 streamByConnection 1
4 globalMute 0
5 ignoreEEPROM 1
6 filePath adi_a2b_system_autoconfig_RJ45.dat
7 start
8 wait
9 getNwInfo
10 pause
11 updateEEPROMAll ..\..\..\cfg\adi_a2b_node_ModuleInfo_1.3.0.0_RJ45.dat 0x51
12 sleep 2000
13 quit
```

Figure 9: EEPROM Update Commands

Note: This is one time process unless there is change in the sub node configuration. The EEPROM programming can be done through SigmaStudio+ as well. Refer to Section 4.1

Another batch file provided is for a quick evaluation of the network and usage of PnP. In the directory "*Targets/software/bin/app/win32*," you'll find a file called "*run_EVAL_RJ45.bat*" that initiates a sequence of commands, from a file located at '*Targets/software/cfg/ini_EVAL_RJ45.txt*' Here is a snip of the file contents.

```
1 %enDbg
2 configType 0
3 streamByConnection 1
4 globalMute 0
5 ignoreEEPROM 0
6 autoApply 1
7 filePath adi_a2b_system_autoconfig_RJ45.dat
8 start
9 wait
10 getNwInfo
11 addCon 0:0 1:0
12 addCon -1:0 0:0
13 applyRouting
14 help
15
```

Figure 10: Evaluation Commands

3.5 Using ADSP-SC59x UART Terminal Application

While using ADSP-SC59x as Host MCU, an interactive terminal application is designed as in the Windows terminal. In this scenario, the SC59x communicates with the PC via UART interface. Here, SC59x receives user commands through the UART terminal and invokes PnP module.

Any terminal emulator program (e.g., TeraTerm) can be used for this evaluation by connecting to the relevant COM port and fixing the baud rate to 115200.

4 Development Workflow

This section outlines the workflow for developing systems with A2B Plug-n-Play. The essential steps involve utilizing SigmaStudio+ for system realization, adapting various software stacks to the platform, maintaining platform-specific libraries, and creating the application. These key steps are discussed in more detail below.

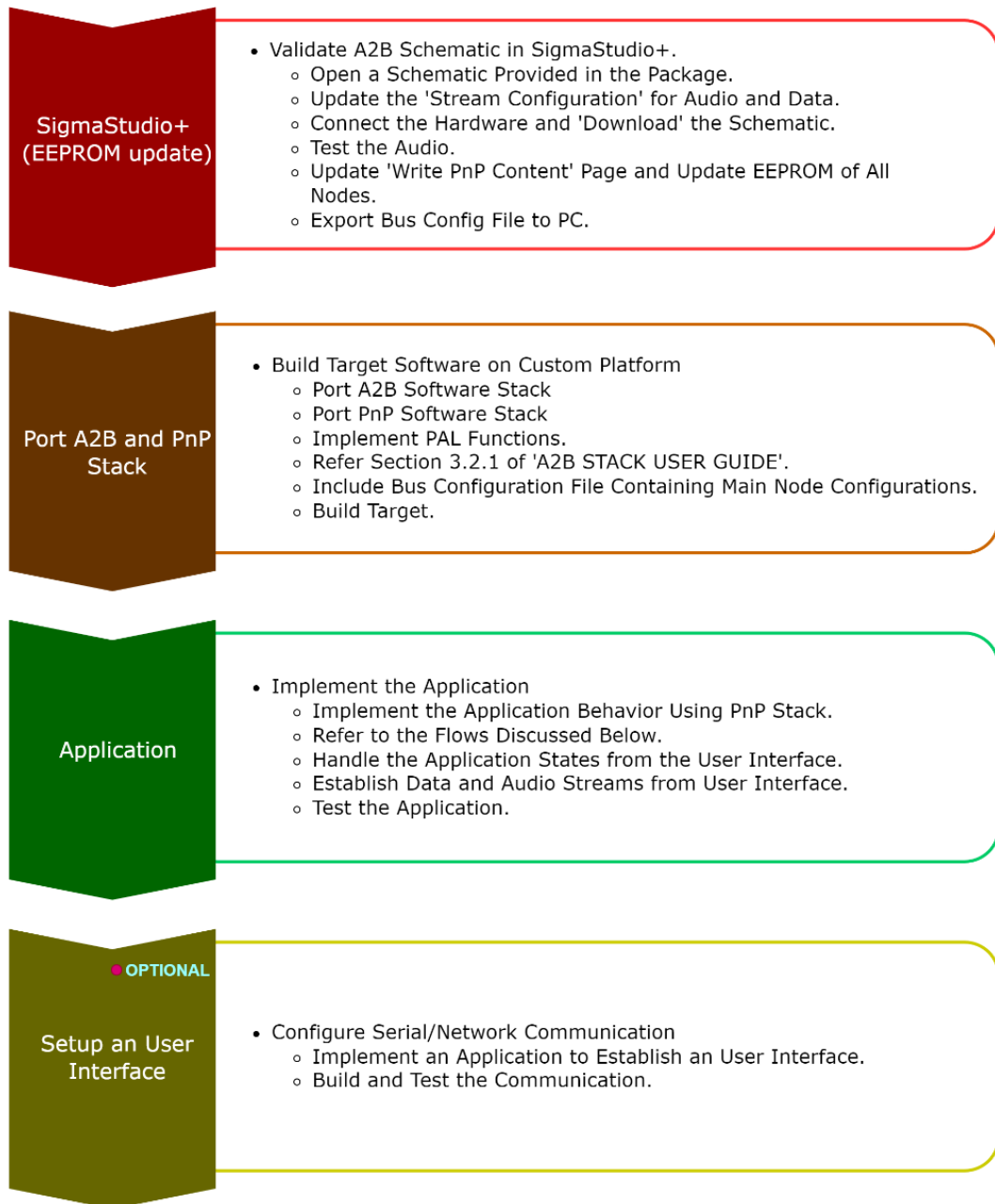


Figure 11: Development Workflow for PnP

4.1 SigmaStudio+ Usage for A2B and PnP

SigmaStudio+ is an effective tool to model and evaluate A2B network with very few quick steps. Any pre-defined/custom platform can be dragged and dropped, configured, connected, and realized on the SigmaStudio+. For A2B it doesn't need a support for MCU and can directly program the platforms. Here is a glimpse of a SigmaStudio+ schematics.

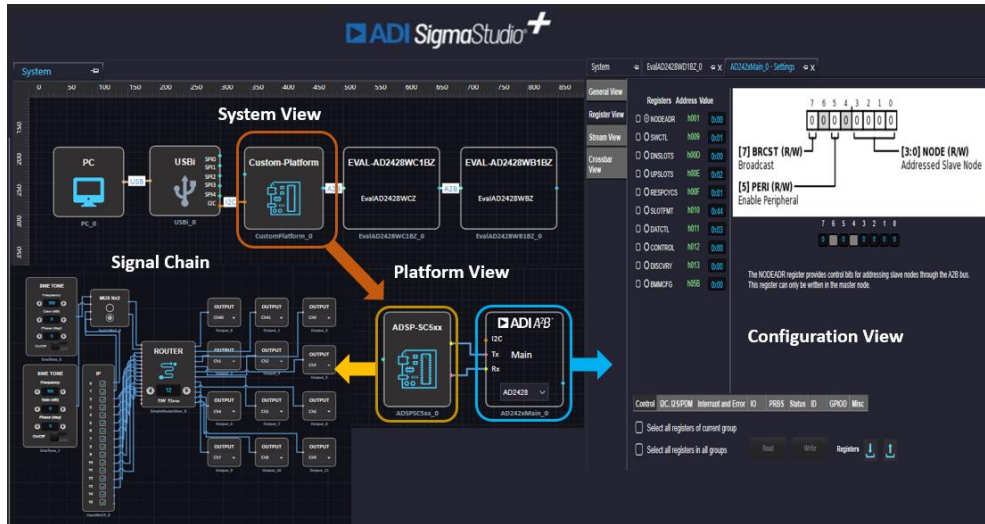


Figure 12: SigmaStudio+ schematics

SigmaStudio+ also has a comprehensive set of windowpanes for different purposes. A labelled diagram is shown below.



Figure 13: SigmaStudio+ Windows

In the above Figure, we see that a schematic is being created using EVAL-AD2437A1NZ and EVAL-AD2437B1NZ hardware which are A2B XLR boards. These are available on the Platforms pane on the left and can be dragged and dropped easily like PC and USBi entity. Once connected Link, Compile and Download option can be used from the top pane to program the hardware.

Some configurations related to A2B network, audio, bandwidth, power, etc. requirements can be done using relevant panes labelled in the Figure. SigmaStudio+ also gives a provision to export the configured settings into various format files which can be used to design final product. These features make SigmaStudio+ an instrumental tool during the ideation state of a product cycle.

With respect to PnP, some pre-defined schematics are provided in the release package as mentioned here. One important task that makes SigmaStudio+ crucial for PnP is programming of EEPROMs of each node. Once we are satisfied with the network's performance on SigmaStudio+ we can download each sub node settings on the EEPROM using this. Here are the steps to carry out for programming the EEPROM.

To run sample demo, PnP content needs to be written to each sub node as shown in Figure 8.

- Open SigmaStudio+ schematics available under “Schematics/adi_a2b_3node_AD2437A1MZ_PnP.ssprj” (RJ45).
- Connect the nodes as shown in Figure 5 and Link-Compile-Download. All the nodes shall be discovered.
- In the schematic, each node is configured to receive and transmit 2 channels of audio.
- Expand the System in the project window and click on the “Write PnP Content” of each node to write the Module information on the EEPROM of each node as shown in Figure 8.

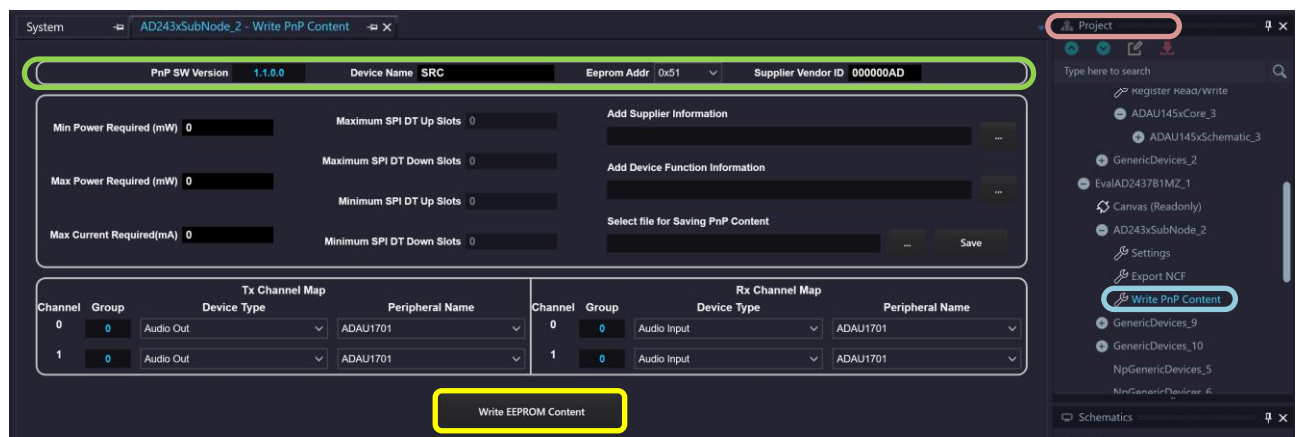


Figure 14: Write PnP Content using Sigma Studio+

- Update Group details in Tx Channel Map and Rx Channel Map. Details of the group-based audio routing can be found in Appendix B.

- Click on the Write button. I2C transactions shall be seen in the Capture window. Since Codec configuration is huge data, screen may freeze for few seconds.
- Repeat the above steps for all the sub nodes in NW.

5 Building PnP for Custom Platform

To establish the A2B PnP system on a custom platform, it is necessary to adapt platform-specific drivers. Additionally, some compile-time parameters should be examined to align with the deployment requirements. The PnP module itself is platform-agnostic. Furthermore, given that the PnP module integrates with the A2B stack, the porting procedure used for the A2B stack can be directly applied to the PnP module, ensuring transition to the custom platform.

To start with, PnP module calls a function “**a2b_pallnit**” where the integrator is expected to provide the function pointers for various drivers (I2C, SPI, Timer etc.). Following sub sections, briefly describe the porting considerations for each of the platform drivers. For the example implementation, please refer to “**adi_a2b_pal.c**”. Also, please refer to a detailed table (Table 3) available in *A2B STACK USER GUIDE (part of A2B plugin for SS+ release package)* explaining their necessities.

Section 5.6 describes the various compile time parameters to be tuned for deployment system.

Once PAL layer is ported and compiled on the target platform, we are ready to integrate an application for the system completion. Section 6 describes the details for application integration.

5.1 I2C

I2C system is responsible for network control. Driver shall allow both burst writes as well as burst read. Note that driver call blocks the A2B Stack and error status shall be passed to stack. I2C can be configured at the one of following rates -100kHz, 400kHz, or 1MHz.

5.2 SPI

Plug and Play module uses SPI only for Data tunnel operation. Hence SPI driver is not mandatory for the platform. When supported, SPI driver should support full-duplex transactions, preferably with DMA.

5.3 Timer

A2B Stack gets timing reference using this driver. One milli-second granularity is sufficient.

5.4 Audio Host

Stack exposes set of functions for configuration of audio host in sync with A2B configuration. Functionality depends upon the system design. If the audio host is external to MCU, it can contain drivers to initialize the subsystem. In the example application, ADAU1452 configuration is carried out, which is the audio host for eval platforms.

5.5 Plugins and Memory Management

A2B stack provides options to load customizable sub-node plugins in a static system. PnP module uses a standard plugin internally. Hence, no need to update any PAL functions related to plugins and memory management.

5.6 Operating System Consideration

PnP module along with A2B stack is flexible to be deployed on bare metal, RTOS and rich OS environments.

5.6.1 Linux and ALSA

PnP module is suitable to be used as user space driver for A2B. Application can maintain an empty codec driver to get the audio into system. The ALSA APIs can be configured as part of the Audio Host driver.

5.7 Compile Time Parameters

5.7.1 Header files

5.7.1.1 `feature.h`

'*feature.h*' contains flags to configure various A2B stack features. The example '*feature.h*' (\software\app\win32\a2bstack-pal\platform\a2b) in the release package sets all the necessary flags required within the A2B stack for supporting PnP operation.

Note: Define *A2B_FEATURE_OPTIMAL_RESPLYCS* if the system requires larger cables (more than 5m).

5.7.1.2 `conf.h`

This header file contains system level configuration options. Please inspect and modify the following macros for the system needs.

- *A2B_CONF_POINTER_SIZE*
- *A2B_CONF_MAX_NUM_MASTER_NODES*
- *A2B_CONF_MAX_NUM_SLAVE_NODES*

5.7.2 Bus Configuration File

For PnP system, only the main node configuration is used from Bus Configuration File (BCF). The remaining information, specific to sub nodes, is read from the sub node EEPROM. If there is no change in the Main node configuration, Bus Configuration File can be exported (from SigmaStudio+) once and reused.

5.7.3 Memory

PnP allocates the memory to A2B stack from the heap section. Please budget ~**10k** bytes heap. Also, budget around **5k** bytes of stack memory. Also, PnP statically allocates ~**12k** bytes per A2B node.

6 Application Integration

Integrating an application is a crucial part for A2B PnP systems. This section describes the PnP Application interface, and callbacks available for application. Also, this section covers integration details with respect to the example application.

Refer to '*A2B_PnP_API_Reference.chm*' for the parameter details.

6.1 Overview of Application and PnP Interaction

Following is a state diagram demarcating the application and PnP states from a high level. On the application start, a state machine is started locally which does basic PnP initialization and triggers the PnP state machine. On the PnP state machine, it starts the network level tasks like discovery, stack management, periodic ticks, etc. and periodically checks for errors. There are various callbacks to the application arising due to various PnP/network events which is also taken care by this state machine. Some error states give back the control to the application state machine for appropriate error handling decision.



Figure 15: App-PnP State Flow

Figure below explains in detail how PnP node discovery mechanism works.

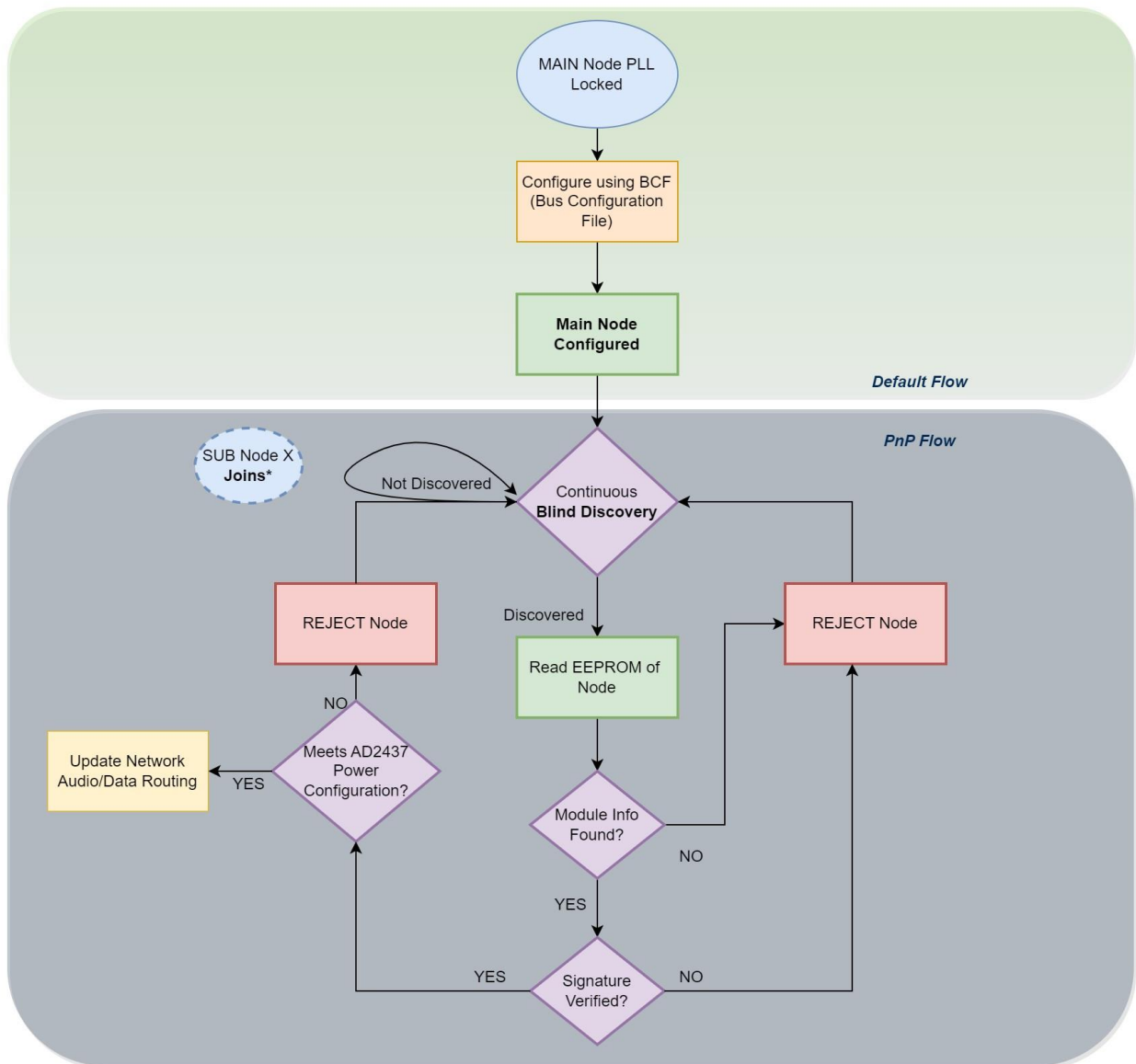


Figure 16: High Level PnP Flow

- Here the flow is shown in 2 blocks *Default Flow* and *PnP Flow*.
- *Default Flow* shows how discovery happens for known nodes from the Bus Configuration File.
- *PnP Flow* shows how discovery is done for newly connected nodes using EEPROM data.
- Application starts a state machine to elegantly handle the above flow of PnP.

- PnP runs a continuous loop of Blind Discovery occurring after a repetitive interval. This checks if there is a new node connected to the network.
- If a new node connects, PnP reads the EEPROM on the node and configures it.
- All these processes are taken care by PnP and application gets notified periodically about the current state of the system.
- To handle the state notifications, application also maintains a state machine currently which handles the callbacks and triggers the PnP APIs whenever needed.
- This state machine is started after the initial system initialization and runs through the lifetime of the system.
- App state machine is also responsible for initiating the PnP initialization, and main node configurations. PnP exposes few APIs for the application to be used explained in next section.

6.2 PnP Module Initialization

When the system powers up and is ready for operation, the first step performed by the application is PnP module initialization. This initialization typically involves a series of APIs that handle the setup and preparation of the PnP module for operation. These APIs configure and prepare the module for Plug and Play functionality:

1. **a2b_pnp_init():** Initializes the PnP parameters to their initial values and initializes connection manager.
2. **a2b_pnp_SetInitParams():** This API takes all the initial parameters set by BCF and application and updates the Bus Description Data to internal structures. It also checks if PnP related set properties are correct.
3. **a2b_pnp_ConfigureMainNode():** This API initializes the Main node audio and data capabilities set by the application.
4. **a2b_pnp_StartDiscovery ():** This API triggers the network discovery based on the configurations set by previous APIs.

6.3 Periodic Tick

a2b_pnp_Task() API drive the internal scheduler and state machine which ultimately realizes all the network management. Application should be designed such that this API gets called more frequently than other APIs or callback handling. Typically, the time elapsed between two successive calls should be within 25 milliseconds. This ensures that the stack is periodically called to keep all the processes/states rolling within the stack which is necessary for the A2B network to function. Note that there is no hard deadline for the periodic call.

In the example application, this API is called from application state machine **app_state_process()** once the state changes to **A2B_PNP_APP_NW_MNG**.

Below is code snippet for reference:

```
406     case A2B_PNP_APP_NETWORK_MNG:                                     a2bapp_common.c
407         /* Monitor a2b network for faults and initiate re-discovery if enabled */
408         a2b_pnp_Task(nw[nwIdx].hPnp);
409         break;
```

Figure 17: Periodic Tick

6.4 Network Control and Status

There are set of APIs for network control and status update which can be handy for the application. The status API set covers a range from high-level network information to a detailed examination of registers. Listing few important APIs, please refer to API guide for the details.

- *a2b_pnp_PauseNewNodeDisc()* – pauses blind discovery.
- *a2b_pnp_ResumeNewNodeDisc()* – resumes blind discovery.
- *a2b_pnp_UpdatePnPModuleInfo()* – updates PnP EEPROM
- *a2b_pnp_RejectNode()* – Rejects newly discovered node.
- *a2b_pnp_MuteNetwork()* – Mute/Unmutes the network audio
- *a2b_pnp_GetNWInfo()* – Gets detailed information about every node in the system
- *a2b_pnp_GetNwBw()* – Gets overall network bandwidth

6.5 Audio Routing

Audio routing in a PnP system is dynamic, allowing users to change or define audio configurations on-the-fly as needed. Notably, PnP audio connections exhibit persistency, which means that these connections endure even if one or both nodes become temporarily disconnected and subsequently reconnect to the system. The system maintains these connections based on unique chip IDs assigned to each A2B transceiver. These connections remain persistent until a complete power cycle of the system is initiated.

Also, there are few design considerations made in PnP to realize the dynamicity of the audio routing. Groups being the most important of them is discussed below along with other details.

6.5.1 Audio Groups

The concept of Group is brought into the PnP audio routing design to ease the process of routing multichannel audio between peripherals of different nodes. 'Group' is a group of channels pre-

defined by the system designer based on the requirement. All the nodes have two set of groups namely, Tx Groups and Rx Groups.

- **Tx Groups:** Groups that accepts audio into the node from other node of A2B network and routes it to the peripherals (audio out, speaker, etc.) attached to it.
- **Rx Groups:** Groups that accepts audio into the node from attached peripherals (microphones, audio in, etc.) and routes it to another node of A2B network.

Suppose there is a node having a 4-channel PDM microphone and it needs to connect to a 4-channel audio codec sitting on another node in the network. In this case a 'Groups' of 4-channels can be created for the PDM microphone (Rx Group) and for the audio codec (Tx Group) during the system design. While routing the audio the Tx Group of latter node can be linked to Rx Group of former node and the audio would be applied. Here user doesn't need to manually link channel-wise connections with multiple commands. Even 4 separate Groups of 1-channel can be formed for the microphone data to be transmitted to different nodes/peripherals accepting 1-channel audio. Another example of this Groups and audio routing is shown in the image below. Here for a better understanding the audio paths are shown outside the A2B paths but in reality all the audio goes through the A2B network only.

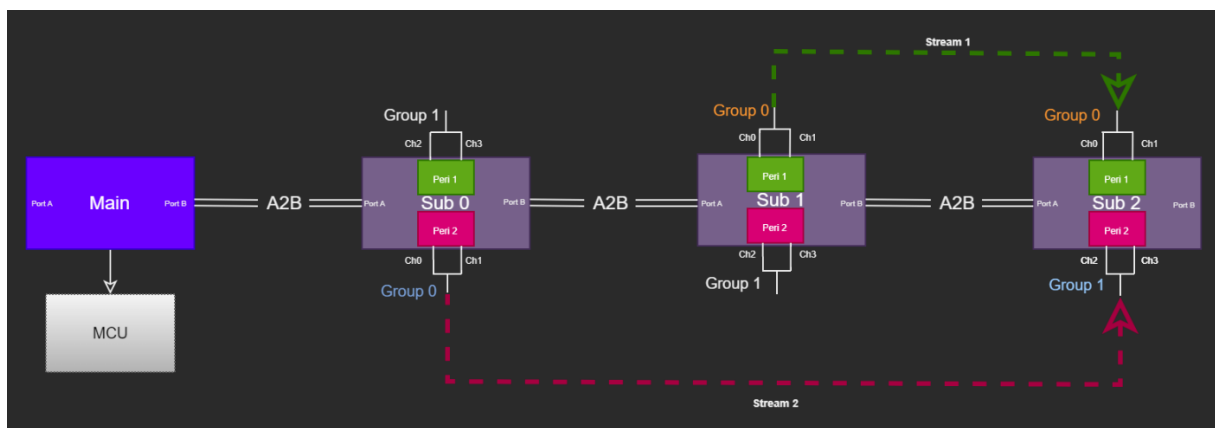


Figure 18: Dynamic Audio Routing

6.5.2 Audio Streams

Audio streams are the connections made between Tx and Rx groups of two nodes. Audio flows from one Rx Group to another Tx Group. Currently, two types of mechanisms are available for establishing a stream, let us see them in detail:

1. Routing using **a2b_pnp_SetAudioRt ()**: In this method, one-to-many connections can be established in one go. User need to specify the source node & group and destination nodes & groups. On the terminal application provided, **addStream** command triggers this method of audio routing. Please refer here for usage details.
2. Routing using **a2b_pnp_AddConByNodeId()**: In this API, individual connection element (within connection matrix) are established in a single call. This API has an advantage where

it can locally remember the connections made between two nodes and can re-establish their connections once they drops and rejoins the network irrespective of their positions in the network. Here user doesn't need to issue routing commands every time a node drops. This can be realized using addCon command using the current release application. Here is an usage example of addCon command:

```
1 %enDbg
2 configType 0
3 streamByConnection 1
4 globalMute 0
5 ignoreEEPROM 0
6 autoApply 1
7 filePath adi_a2b_system_autoconfig_RJ45.dat
8 start
9 wait
10 getNwInfo
11 addCon 0:0 1:0
12 addCon -1:0 0:0
13 applyRouting
14 help
15
```

Figure 19: addCon Command

addCon command is used twice to establish connections between node 0 to node 1 and node -1(Main) to node 0. The format as seen from figure is addCon <source node>:<Rx Group> <Destination node>:<Tx Group>. Similarly, we have API to remove single connection (a2b_pnp_RemoveConByNodeId)

Note: Recommended to use connection based API in the complex UI system as it provides more flexibility.

6.5.3 Apply Routing

In the Figure18, we observe the utilization of a command named "applyRouting." This command internally triggers the API function, namely "*a2b_pnp_ApplyRouting*." This API serves the purpose of applying the routing configuration settings to the hardware by updating the pertinent slot registers of each node in the system. The effectiveness of audio routing established through "addCon" or "addStream" takes place only when this API is invoked, making the audio routing fully operational within the network. Note that Routing can be applied only when PnP module is not busy with blind discovery (search for new node)

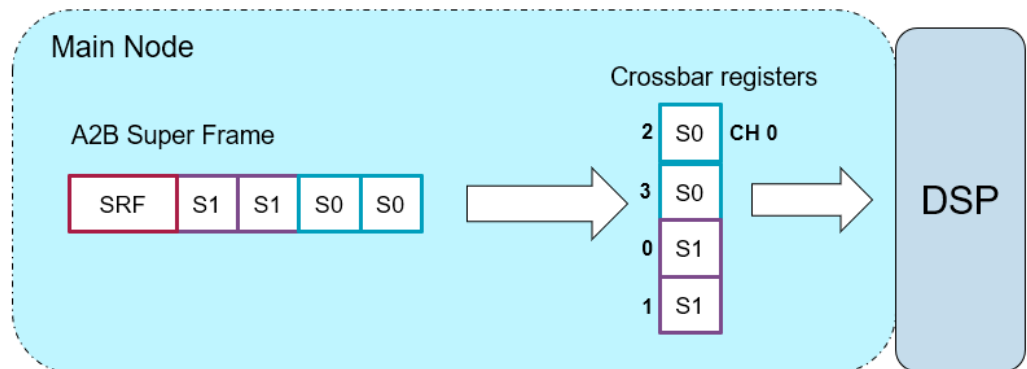
Each time this API is invoked, the Plug and Play (PnP) system recalculates the A2B slot configurations for every node and synchronously applies these configurations to all nodesThe **a2b_pnp_ApplyRouting()** API also takes care of the data tunnel slots in the A2B slot calculation process.

6.5.4 Impact of Network Events on Audio Routing

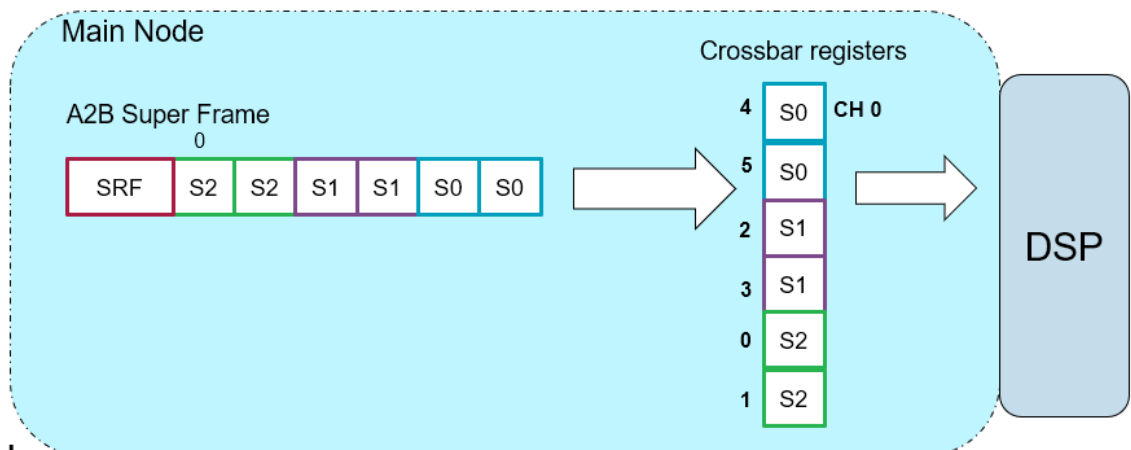
Network events, like establishing a new audio connection or removing a node, can affect the distribution of existing audio. PnP selectively mutes the audio for short duration when there's a change in network routing requirements, particularly in the A2B Tx crossbar registers, as compared to the current routing. In simpler terms, if a node is receiving audio from multiple A2B nodes, altering the source nodes will impact the ongoing audio.

Let's consider a basic example of two sub-nodes contributing audio to a main node. PnP rearranges the audio channels based on the application's desired channel order. In this scenario, PnP ensures that the audio from the sub node 0 is placed ahead of Sub node 1. Now, if a new node is added to the network, and the application requests a new routing configuration, PnP needs to reorganize the audio channels using the crossbar registers. Since cross bar registers cannot be updated simultaneously with slot registers, PnP mutes the audio while applying the new slot configuration.

2 nodes



Once new Device connected and adds audio



3 nodes

Figure 20: Audio Routing Limitation

It's important to note that in some situations, when new routing requests only involve changes within the network slot registers and do not affect the crossbar registers, there is no need to mute the audio. In such cases, the new configuration can be applied seamlessly without any disruption to the audio.

6.5.5 Application Call-backs

PnP notifies the application about the impending mute. This allows the application to synchronize other DSP operations. Similarly, PnP notifies the application after unmuting the audio. Following events are used for notification.

- **A2B_PNP_AUDIO_MUTE**
- **A2B_PNP_AUDIO_UNMUTE_ROUTING_DONE**

Additionally, PnP offers configuration option for network wide mute during the crossbar update.

6.6 Data tunnel

Data tunnels are dedicated upstream and downstream A2B bus data slots used to transfer SPI data in SPI over distance communication. There are 2-12 slots for upstream transfer and 2-12 slots for downstream transfer. The transceiver initiating the SPI over distance transaction is the *tunnel owner*, and the transceiver receiving the SPI over distance transaction is the *tunnel responder*. In following figure, the data tunnel topology is being shown:

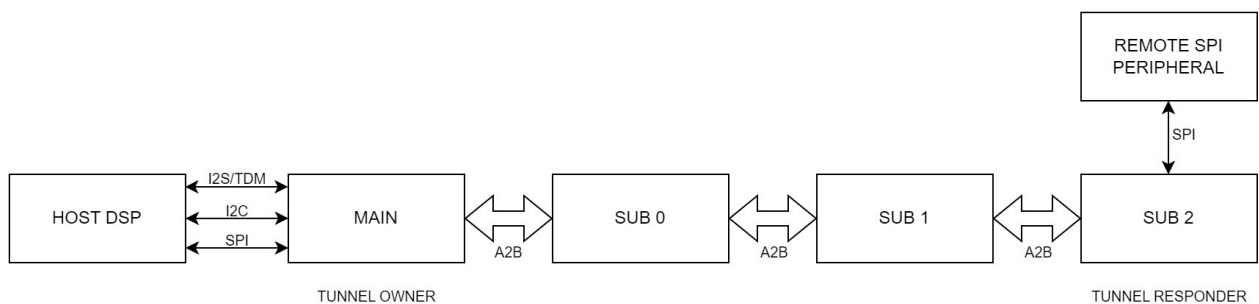


Figure 21: Data Tunnel Topology

Currently, in PnP all the nodes participate for data tunnel if enabled.

Data tunnel for the nodes can be enabled from SigmaStudio+ Stream Configuration window as shown below.

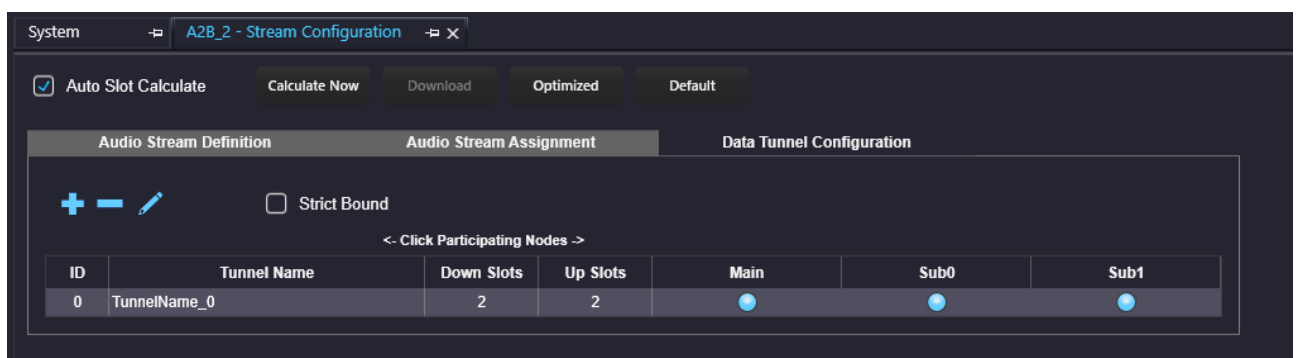


Figure 22: Data Tunnel Configuration

On the application side, a data tunnel can be established by the API **a2b_pnp_SetSPIDTRt()** and it can also be configured by using **addTunnel** command in the terminal application.

```

605  else if (!strcmp((const char*) g_cCommand, "addTunnel"))
606  {
607      sscanf(aStr, "%s %i", g_cCommand, &nArg1 );
608      a2b_pnp_GetDefaultDTStream(nw[nwIdx].hPnp, &nw[nwIdx].DTStream, &num);
609      a2b_pnp_SetSPIDTRt(nw[nwIdx].hPnp, &nw[nwIdx].DTStream, 1, true);
610  }
611

```

Figure 23: addTunnel Command

Here **a2b_pnp_GetDefaultDTStream()** is used to configure a default DT stream and store that in a local **DTStream** data structure. Consequently **a2b_pnp_SetSPIDTRt()** is used to update the **DTStream** data to actual network data structure.

Note: **applyRouting** command is required to apply the tunnel on the A2B bus data slots.

6.7 GPIO over Distance

The GPIO over distance feature allows GPIO communication to occur over the A2B bus without host intervention after initial programming.

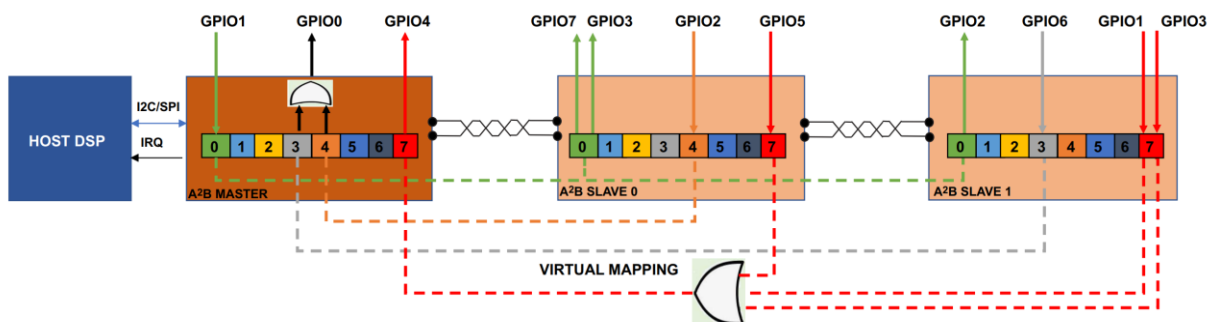


Figure 24: GPIO over Distance

The *GPIO Over Distance* figure shows virtual mapping using GPIO pins. Multiple virtual ports are configured with IOs mapped to and from different transceivers. Data from an input GPIO pin connected to a virtual port is reflected to the output GPIO pin connected to same port on a different A2B node.

PnP has APIs for setting up and clearing the GPIO over Distance.

a2b_pnp_SetupGPIOOverDistance() is for setting up a GPIO over Distance signalling and

a2b_pnp_ClearGPIOOverDistance() is for clear the signalling. Please refer API Guide for usage details.

6.8 Callbacks

Application handles all the notifications from the PnP in Callback functions. Currently, three callbacks are implemented on the application each intended to handle a set of specific events.

6.8.1 PNP CALLBACK

This callback handles the A2B network events and the audio routing level of processes. This callback is triggered by PnP at onset of relevant events. Here is the list of events handled by this callback:

- **A2B_PNP_NEW_NODE_CONNECTED**
- **A2B_PNP_NODE_DISCONNECTED**
- **A2B_PNP_NODE_REJECTED**
- **A2B_PNP_NW_REDISCOVERY_INITIATED**
- **A2B_PNP_BD_COMPLETE_WITH_NEW_NODE**
- **A2B_PNP_HW_FAULT**
- **A2B_PNP_APPLY_AUDIO_ROUTING**
- **A2B_PNP_BASIC_AUTH_FAIL**
- **A2B_PNP_AUDIO_MUTE**
- **A2B_PNP_AUDIO_UNMUTE_ROUTING_DONE**
- **A2B_PNP_MAIN_PLL_LOCK_FAILURE**
- **A2B_PNP_I2C_FAILURE**
- **A2B_PNP_NO_VALID_RESP_CYCS**

6.8.2 ERROR CALLBACK

This callback is designed to handle all the PnP level of error that might be generated during the operation of application.

- **A2B_PNP_NULL_HANDLE_ERR**
- **A2B_PNP_BCF_FILE_OPEN_ERR**
- **A2B_PNP_MODULE_CATCH_ALL_ERR**

6.8.3 INTERRUPT CALLBACK

This callback is designed to handle IO interrupts generated by A2B sub nodes.

- A2B_PNP_INTTYPE_IO0PND
- A2B_PNP_INTTYPE_IO1PND
- A2B_PNP_INTTYPE_IO2PND
- A2B_PNP_INTTYPE_IO3PND
- A2B_PNP_INTTYPE_IO4PND
- A2B_PNP_INTTYPE_IO5PND
- A2B_PNP_INTTYPE_IO6PND
- A2B_PNP_INTTYPE_IO7PND

6.9 Multi-Main Considerations

A2B software stack supports the design of multiple main node configuration which spawns two A2B network and gets controlled by one host. See the Figure below for a reference:

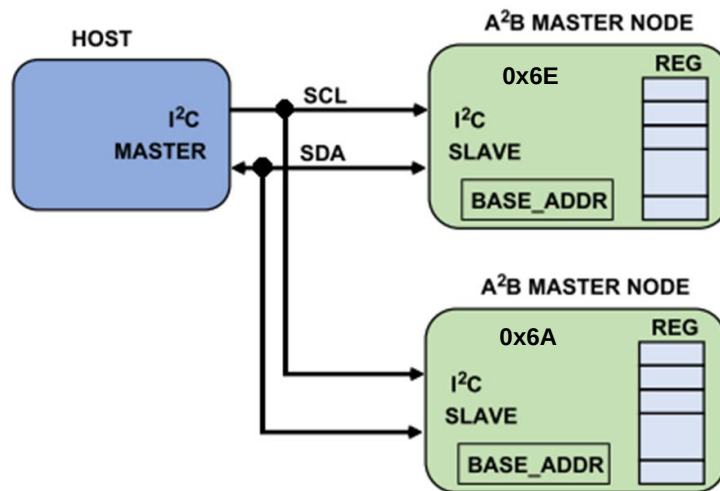


Figure 25: Multi-Main Design

PnP is capable of handling multi-main design where two separate A2B networks can be driven from the same Host using I2C bus. As the configuration of A2B networks happens from same host over I2C, each Main node should be configured with different I2C slave addresses. In the provided schematics case one AD2437 main has the I2C address of 0x6E and another AD2437 main has the default I2C address of 0x6A.

On the application usage, if using multi-main system, PnP software should be initialized and handled separately for both the A2B networks. Currently, two separate application state machine is called for exclusiveness. Here is a Figure showing the code snippet having two **app_state_process()** calls:

```

129 //Thread for PnP execution. State based representation.
130 DWORD WINAPI pnp_thread(LPVOID lpParam) { a2bapp_server.c
131     PipeThreadArgs* thread_args = (PipeThreadArgs*)lpParam;
132
133     while (1) {
134         app_state_process(0);
135         if(enable_flag == 1)
136         {
137             ExecCmd();
138             enable_flag = 0;
139         }
140
141     #if A2B_CONF_MAX_NUM_MASTER_NODES > 1
142         app_state_process(1);
143     #endif
144     /*-----*/
145     /* Add your other continuous monitoring application code here */
146     /*-----*/
147
148     /* (DEBUG purpose) set eEnableDisc to resume the paused discovery after node rejection */
149
150 }
151
152 return 0;
153 }

```

Figure 26: Multi-Main Process

Inside each **app_state_process()** an application flag **bUserReady** is used to trigger PnP and network discovery operation. This flag is enabled by the terminal command **start**.

```

331 if (!strcmp((const char*) g_cCommand, "start")) cmd_parse.c
332 {
333     sscanf(aStr, "%s %i %i", g_cCommand, &nArg1, &nArg1);
334     nw[nwIdx].bUserReady = 1;
335     ADI_UART_PRINT("\r\n");
336 }
337

```

Figure 27: start Command

As shown in above Figure, **bUserReady** is a flag in a structure array **nw** with **nwIdx** as its index. The index **nwIdx** decides the network we want to configure. By default, this variable is set to 0. User can use **setNetwork 0/1** command to update this index value.

```
342     else if (!strcmp((const char*) g_cCommand, "setNetwork"))
343     {
344         sscanf(aStr, "%s %i", g_cCommand, &nArg1);
345         nwIdx = nArg1;
346     }
```

cmd_parse.c

Figure 28: setNetwork Command

Setting this *nwIdx* correctly before issuing start command is necessary for discovery intended network. This index on application translates down to the network chains specified in Bus Config File exported from SigmaStudio+. Here is how it is correlated with BCF:

```
57  /* BUS CONFIGURATION */
58  ADI_A2B_BCD sBusDescription =
59  {
60      /* Number of master nodes/daisy chains in the network */
61      .nNumMasterNode = 2u,
62
63      /* Array of Master-slave daisy chains */
64      .apNetworkconfig = { &sChainConfig0, &sChainConfig1 },
65
66      /* Network control guidance */
67      .sTargetProperties =
68      {
```

adi_a2b_multimain.c

Figure 29: Multi-main Config File

Note: User should make sure the I2C initialization and Audio host configuration should be done only once.

7 System Details for ADSP-SC59x Platform

7.1 System Requirements

7.1.1 Software Requirements

- CrossCore Embedded Studio v2.11.0 or above. This can be downloaded from www.analog.com/cces
- [SigmaStudio+ v2.1.0.](#)

7.1.2 Hardware Requirements

- A2B Evaluation Board configured as Main
 - EVAL-AD2437A1NZ Rev B or EVAL-AD2437A1MZ
 - 24V Power Supply
 - XLR Cable with connectors.
 - EVAL-ADUSB3EBZ (USBi) board with USB cable (1.7m or 10m)
- Sub nodes
 - EVAL-AD2437B1NZ Rev B or EVAL-AD2437B1MZ
 - XLR Cables with connectors.
 - Stereo Speakers/headsets.

7.2 Hardware Setup - XLR

7.2.1 Evaluation boards

The Evaluation boards used in the AD2437 demos are explained in the following subsections.

7.2.2 EVAL-AD2437A1NZ

Figure 30 shows an EVAL-AD2437A1NZ Rev. B board which can be used as either an A2B main or local powered sub node. The board has following components

- AD2437 A2B transceiver
- ADAU1452 Sigma-DSP Audio Processor

- 512K Self-Boot Memory (EEPROM)
- S/PDIF Coaxial connectors
- Codec AD1937
- 4 Audio channel XLR Inputs

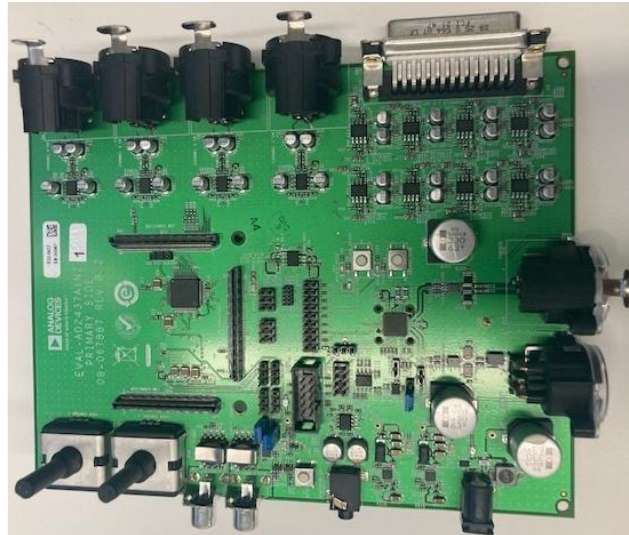


Figure 30: A2B EVAL-AD2437A1NZ board

Table 2: Jumper Settings

Jumper	Purpose in EVAL-AD2437A1NZ	Main	Jumper	Purpose in EVAL AD2437B1NZ	Sub x
JP1	Self Discovery Mode	1-2	P11	DRX0 source select	2-3
JP3	I2C ADR1	x			
JP4	I2C ADR2	x			
JP5	Boot Disable	1-2			
JP6	Clock Select	1-2			
P1	XLR Channel IN1	Open			
P2	I2S Probe Head	Open			
P3	XLR Channel IN3	Open			
P4	XLR Channel IN4	Open			
P5	ADAU1452 I2C/SPI	Open			
P22	Sigma Studio+	Open			

P7	ADAU1452 SDATA_IN2	Open			
P8	ADAU1452 SDATA_OUT2	Open			
P9	XLR Channel IN2	Open			
P23	SPI Probe Head	Open			
P25	Line Out	Connect			
P16,P20,P15	Som Connector	Connect			
J9	DB25	Open			

7.2.3 EVAL-AD2437B1NZ

The Figure 31 shows an EVAL-AD2437B1NZ board which can be used as an A2B sub node. The board has following peripherals

- AD2437 A2B transceiver
- 4 PDM Microphones
- Class D Amplifier BD28623MUV
- Codec ADAU1701
- 512K Self-Boot Memory (EEPROM)



Figure 31: A2B EVAL-AD2437B1NZ board

7.2.4 EV-SC594-SOM Rev B

Figure 32 shows EV-SC594-SOM evaluation board which is used with EVAL-AD2437A1NZ board. It is used as an audio host to the main node. It consists of the following peripherals.

- 8 Gb DDR3 (800 MHz) Memory
- 512 Mb Quad SPI Flash Memory
- 256 Mb Octal SPI Flash Memory

- FTDI USB-to-UART

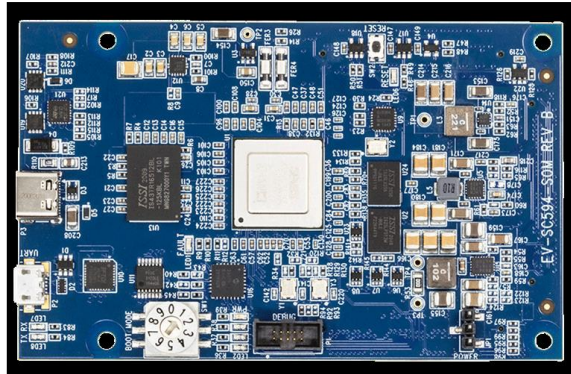


Figure 32: EV-SC594-SOM Rev B (Top view)

7.3 Connections

To run the sample demo, the following setup connections are to be made.

7.3.1 Evaluation boards

A2B Evaluation boards can be connected in any order (Plug and Play use case)

- Mount EV-SC594 SOM on Eval-AD2437A1NZ(Main) board. Attach JTAG to P1 on EV-SC594-SOM board and other end of JTAG to USB port of PC.
- Connect XLR cable between the A2B connector on the Main board and the “A” connector on the Sub 0 board.
- Connect XLR cable between the “B” connector on the Sub 0 board and the “A” connector on the Sub 1 board. Same connections to be repeated for all the other boards
- Connect 24V 3A power supply to the power connector (P14) on AD2437A1NZ/ AD2437A1MZ board.

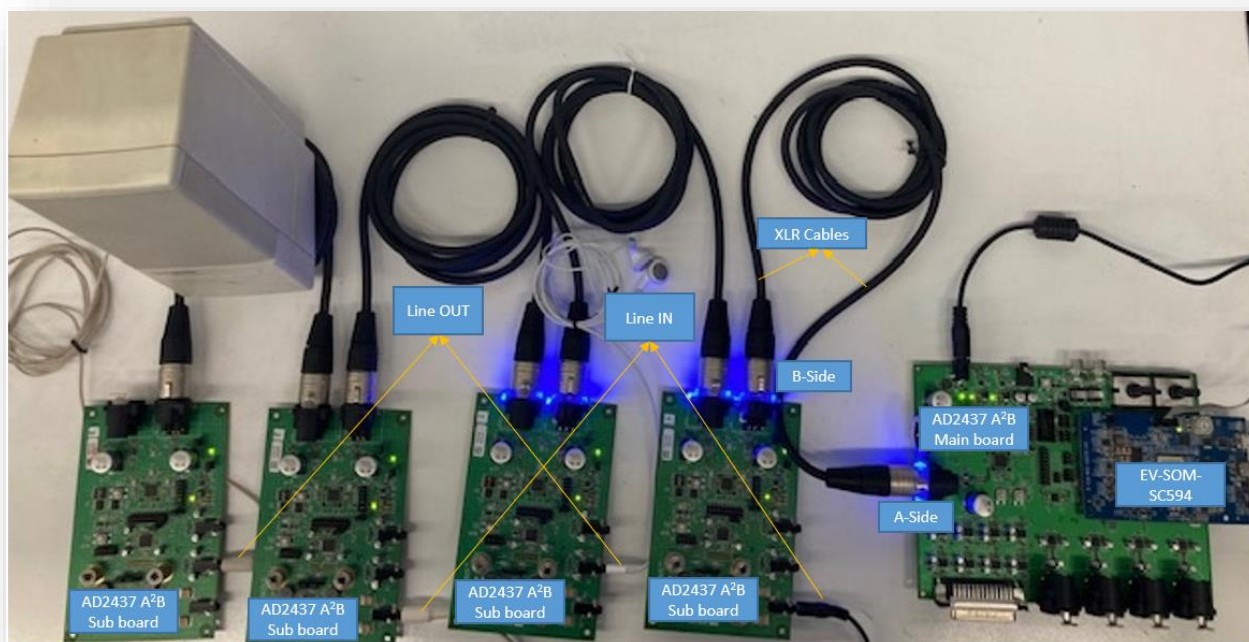


Figure 33: Eval Board Connections

7.3.2 USBi

- Attach a USBi cable to the main board connector P22. Other end of the cable should be connected to the PC. The connection is also used to directly control A2B system when PC is used as the target processor.
- This connection is required to write EEPROM content on the sub nodes. Refer section 7.4.1 . adi_a2b_3node_AD2437A1NZ_PnP.ssprj schematics can be used for the same.

7.3.3 Audio In/out

- Depending on the configured functionality of the device, connect Headphones/speakers to P7(3.5mm headphone jack) or Audio source to P4(3.5mm headphone jack) on AD2437B1NZ.

7.4 Running the Plug and Play Demo - XLR

7.4.1 Write PnP Content on the sub nodes using SigmaStudio+

To run sample demo, PnP content needs to be written to each sub node as shown in **Figure 34**.

- Open SigmaStudio+ schematics available under “Schematics/adi_a2b_3node_AD2437A1NZ_PnP.ssprj”.
- Connect the nodes as shown in Figure 33: Eval Board Connections and Link-Compile-Download. All the nodes shall be discovered.
- In the schematic, each node is configured to receive and transmit 2 channels of audio
- Expand the System in the project window and click on the “Write PnP Content” of each node to write the Module information on the EEPROM of each node as shown in Figure 34.

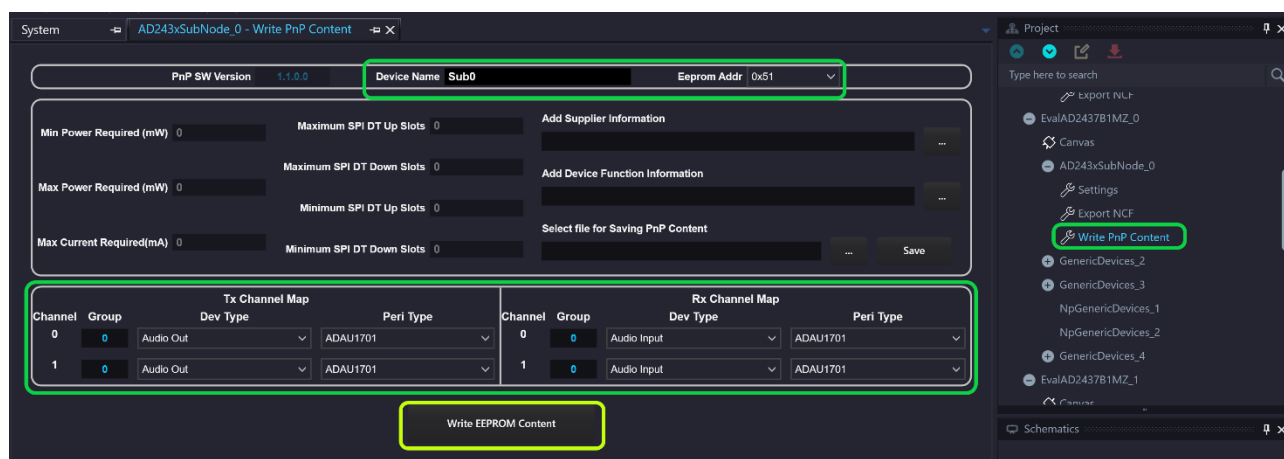


Figure 34: Write PnP Content using Sigma Studio+

- Update Group details in Tx Channel Map and Rx Channel Map. Details of the group-based audio routing can be found in Appendix B.
- Click on the Write button. I2C transactions shall be seen in the Capture window. Since Codec configuration is huge data, screen may freeze for few seconds.
- Repeat the above steps for all the sub nodes in NW

7.4.2 Running the demo

7.4.2.1 HW Setup

- Remove the SigmaStudio Connector from the Main node and mount EV-SC594-SOM Rev B Eval board on it. Connect a JTAG to it.
- Connect a USB cable between host PC and USB-UART connector (P2) on EV-SC594-SOM
- To run the demo, minimum one main node and one Sub node shall be connected before *running the Target application*.

7.4.2.2 Running the Target Demo application

- Open CCES and import "Target\\examples\\demo\\ a2b-adsp-sc59x_AD2437-XLR_PnP application. Compile and build the application.
- Open Tera Term and select the USB Serial Port. Go to Setup -> serial port and change the speed to 115200. All the print messages will appear here.
- Connect Two sub nodes as suggested in section 7.3.1 . Run the application and you would see that existing nodes in the Network are detected dynamically and the functional capabilities are printed, by reading the PnP Content written on the EEPROM. UART output is shown in Figure 35.
- Try connecting new sub nodes or remove existing sub nodes Software shall be able to discover them dynamically and configure them as per the device configuration on EEPROM
- Define Audio connections/streams as desired.

```

          #####
        #####
      #####
    #####
  #####
#####

=====
|                                     |
|               A2B Plug and Play Application               |
|               Build Time : Jun 1 2023, 20:22:50           |
|                                     |
=====

A2B>
Node Connected: 0

Main Node <-1>Device Name: MAIN
Data Tunnel: CAN BE USED
Supported Device Type(s):  AUDIO OUT  AUDIO IN
Tx Group 0 Name: G0
Group Channels: 0 1
Rx Group 0 Name: G0
Group Channels: 0 1
Sub Node 0 - Device Name: SNK
Data Tunnel: CAN BE USED
Supported Device Type(s):  AUDIO OUT  AUDIO IN
Tx Group 0 Name: ADAU1701
Group Channels: 0 1
Rx Group 0 Name: ADAU1701
Group Channels: 0 1
*****Audio Routing Details*****
No Audio Streams defined
Press
A: Add new stream
R: Redefine the complete Audio Routing
I: Ignore
Node Connected: 1

Main Node <-1>Device Name: MAIN
Data Tunnel: CAN BE USED
Supported Device Type(s):  AUDIO OUT  AUDIO IN
Tx Group 0 Name: G0
Group Channels: 0 1
Rx Group 0 Name: G0
Group Channels: 0 1
Sub Node 0 - Device Name: SNK
Data Tunnel: CAN BE USED
Supported Device Type(s):  AUDIO OUT  AUDIO IN
Tx Group 0 Name: ADAU1701
Group Channels: 0 1
Rx Group 0 Name: ADAU1701
Group Channels: 0 1
Sub Node 1 - Device Name: SRC
Data Tunnel: CAN BE USED
Supported Device Type(s):  AUDIO OUT  AUDIO IN
Tx Group 0 Name: G0
Group Channels: 0 1
Rx Group 0 Name: G0
Group Channels: 0 1

```

Figure 35: UART Output - Connection

- Existing node can also be disconnected dynamically from the Network. Refer to Figure 36

```

Bus Drop Detected @ Node: -1

Press
A: Add new stream
R: Redefine the complete Audio Routing
I: Ignore

Node Connected: 0

Main Node <-1>Device Name: MAIN
Data Tunnel: CAN BE USED
Supported Device Type(s):  AUDIO OUT  AUDIO IN
Tx Group 0 Name: G0
Group Channels: 0  1
Rx Group 0 Name: G0
Group Channels: 0  1
Sub Node 0 - Device Name: SNK
Data Tunnel: CAN BE USED
Supported Device Type(s):  AUDIO OUT  AUDIO IN
Tx Group 0 Name: ADAU1701
Group Channels: 0  1
Rx Group 0 Name: ADAU1701
Group Channels: 0  1
*****Audio Routing Details*****
No Audio Streams defined

Press
A: Add new stream
R: Redefine the complete Audio Routing
I: Ignore

Node Connected: 1

Main Node <-1>Device Name: MAIN
Data Tunnel: CAN BE USED
Supported Device Type(s):  AUDIO OUT  AUDIO IN
Tx Group 0 Name: G0
Group Channels: 0  1
Rx Group 0 Name: G0
Group Channels: 0  1
Sub Node 0 - Device Name: SNK
Data Tunnel: CAN BE USED
Supported Device Type(s):  AUDIO OUT  AUDIO IN
Tx Group 0 Name: ADAU1701

```

Figure 36: UART Output - Disconnection

- Once nodes are connected, user can provide the Audio Stream configuration through UART.

7.5 Considerations for RJ45 Evaluation board Set up

The steps to run RJ 45 Evaluation board set up is similar to XLR. This section describes the additional considerations with respect to RJ 45 system.

7.5.1 Hardware Setup – RJ45

7.5.1.1 Evaluation boards

The Evaluation boards used in the AD2437 demos are explained in the following subsections.

7.5.1.1.1 EVAL-AD2437A1MZ

Figure 30 shows an EVAL-AD2437A1MZ board which can be used as either an A2B main or local powered sub node. The board has following components

- AD2437 A2B transceiver
- ADAU1452 Sigma-DSP Audio Processor
- 512K Self-Boot Memory (EEPROM)
- S/PDIF Coaxial connectors
- Codec AD1937

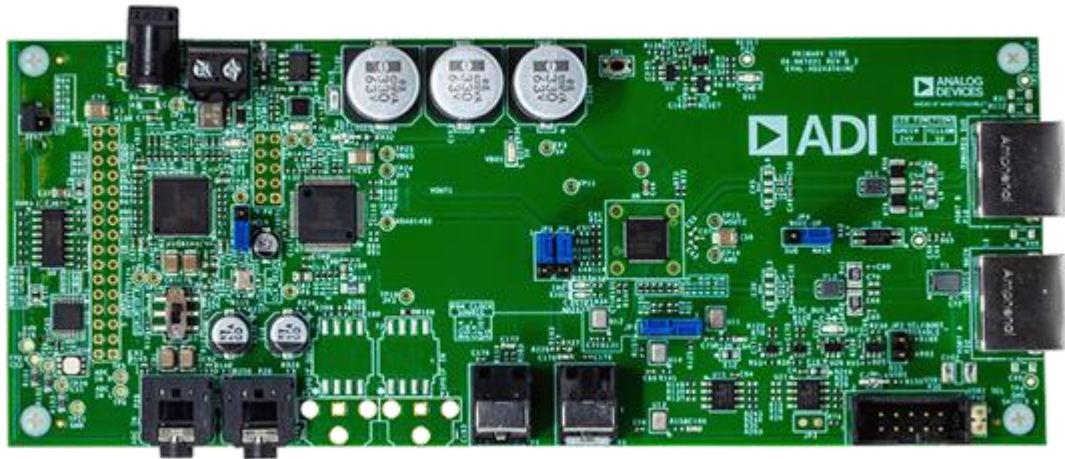


Figure 37: A2B EVAL-AD2437A1MZ board

Table 3: Jumper Settings

Jumper	Purpose in EVAL-AD2437A1MZ	Main	Jumper	Purpose in EVAL AD2437B1MZ	Sub x
P4	WAKE UP	1-2	P3	BCLK	Installed
P7	PDM_CLK	2-3	P16	PDM CLK	Installed
JP6	SIO0 Routing	1-2	P4	SIO0 Routing	2-3

JP9	SIO1 Routing	1-2	P23	SIO1 Routing	2-3
JP38	ADAU1452 CLK Select	1-2	P18	I2C Bus Switch	Open
JP47	PWM Select	Open	P22	Boot DISABLE	Open

7.5.1.1.2 EVAL-AD2437B1MZ

The Figure 31 shows an EVAL-AD2437B1MZ board which can be used as an A2B sub node. The board has following peripherals

- AD2437 A2B transceiver
- 4 PDM Microphones
- Class D Amplifier BD28623MUV
- Codec ADAU1701
- 512K Self-Boot Memory (EEPROM)



Figure 38: A2B EVAL-AD2437B1MZ board

7.5.1.1.3 EV-SC594-SOM Rev B + EV-SOMCRR-EZKIT

Figure 32 shows EV-SC594-SOM evaluation board which is used with EVAL-AD2437A1MZ board. It will be used as an audio host to the main node. It consists of the following peripherals.

- 8 Gb DDR3 (800 MHz) Memory
- 512 Mb Quad SPI Flash Memory

- 256 Mb Octal SPI Flash Memory
- FTDI USB-to-UART

7.5.1.1.4 EV-SOMCRR-EZKIT Rev A

Error! Reference source not found. shows the EV-SOMCRR-EZKIT which can be used with EV-21569-SOM or EV-SC594-SOM. This board along with a processor SOM board will be used as an audio host to the ADZS2435-MINI. It consists of the following peripherals.

- ADAU1979 – Quad Analog-to-Digital Converter
- ADAU1962A – 12 Channel, 192kHz, 24-bit DAC
- CAN FD 2× ADM3056E
- Macronix MX66LM1G45G 1 Gbit OSPI Flash Memory
- Gigabit Ethernet TI DP83867
- 10/100 Ethernet TI DP83848
- USB 2.0 PHY Microchip USB3340
- USB 2.0 to QSPI FTDI
- MicroSD Card Socket

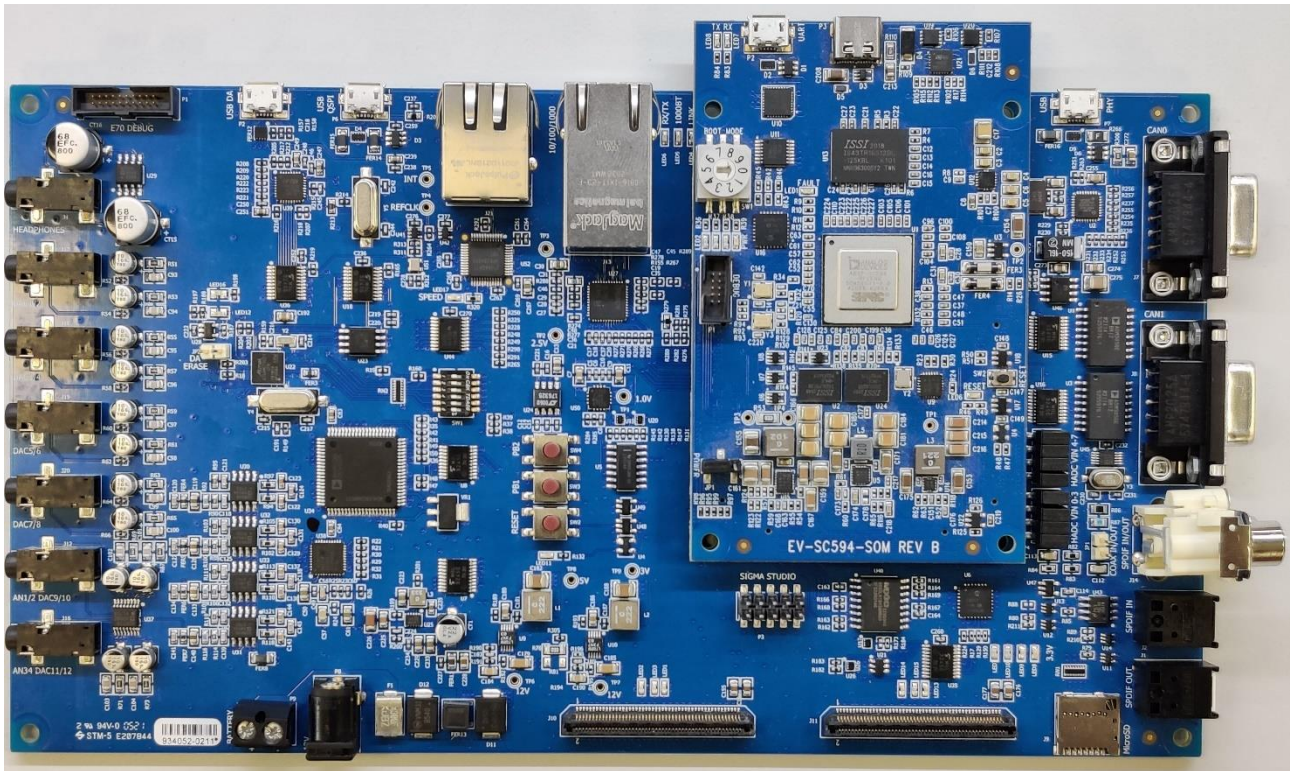


Figure 39: EV-SOMCRR-EZKIT + EV-SC594-SOM

7.5.2 Connections

To run the sample demo, the following setup connections are to be made.

7.5.2.1 Evaluation boards

The hardware connections to be done are described below.

1. SC594 SOM is mounted on Eval-AD2437A1MZ(Main) board.
2. A2B Evaluation boards shall be connected in the following order

EV-SC594-SOM (Rev B) <-> Eval-AD2437A1MZ(Main) <-> Eval-AD2437B1MZ(Sub 0) <-> Eval-AD2437B1MZ(Sub 1) as shown in **Error! Reference source not found.**

3. Connect RJ45 cable between the “B” connector on the Main board and the “A” connector on the Sub 0 board. Repeat the same connection between Sub 0 and Sub 1
4. Connect 24V power supply to the power connector (P14) on Eval-AD2437A1MZ(Main) board.
5. Attach JTAG to P1 on EV-SC594-SOM board and other end of JTAG to USB port of PC.

6. Connect Audio source on J12 on EV-SOM-CRR board.
7. Connect Audio Source on P7 on AD2437B1MZ Sub 0 board.
8. Connect audio sink (e.g., Headphones/speaker) to J4 on EV-SOM-CRR board.
9. Connect separate audio sink (e.g., Headphones/speaker) to P19 (3.5mm headphone jack) on AD2437B1MZ board (Sub 1).



Figure 40: Eval Board Connections

7.5.2.2 USBi

- Attach a USBi cable to the main board connector JP42. Other end of the cable should be connected to the PC. The connection is also used to directly control A2B system when PC is used as the target processor.
- This connection is required to write EEPROM content on the sub nodes. Refer section 7.4.1 . adi_a2b_3node_AD2437A1MZ_PnP.ssprj schematics can be used for the same.

7.5.2.3 Audio In/out

Depending on the configured functionality of the device, connect Headphones/speakers to P7(3.5mm headphone jack) or Audio source to P4(3.5mm headphone jack) on AD2437B1MZ.

8 Terminology

Table 3: Terminology

Term	Description
BCF	Bus Configuration File
DSP	Digital Signal Processor
USB	Universal Serial Bus
USBi	USB to SPI/I2C converter.
SPI	Serial Peripheral Interface
I2C	Inter IC Communication
PAL	Platform Abstraction Layer
PNP	Plug and Play

9 References

Table 4: References

Reference No.	Description
1	Stack User Guide
2	PnP API Guide
3	SS+ guide